



TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: 151287

Need help? Post on our [class slack channel](#) or email us at support@codepath.org

[Getting Started](#)[Overview](#)[Session #1](#)[Session #2](#)[Assignment](#)[Cheatsheet](#)[IDE Setup](#)[Learning with AI](#) ✨[Learning with AI](#)[Resources](#)[Report](#)[▶ Course Info](#)[Course Progress](#)[Unit 1](#)[Unit 2](#)[Unit 3](#)[Unit 4](#)[Unit 5](#)[Unit 6](#)[Unit 7](#)

Unit 1 Cheatsheet

Here's a quick reference sheet for Unit 1. While not an exhaustive list, it highlights the key syntax and concepts you'll use in this unit, plus a few optional ideas that may help with problem-solving. You're still expected to know required material from earlier units.

Sections are labeled for clarity:

- In-Scope: May appear on the assessment
- Not In-Scope: Useful, but not required



Unit Goals

- ☐ Write, use, and modify variables, functions, and print statements
- ☐ Use conditional logic (if statements)

Unit 8

Unit 9

Unit 10

Submission
Guide

☐ Work with lists (accessing, iterating, etc)

▼ General Concepts ☒ In-Scope

Built-In Functions

Print

`print(s)` Prints the message `s` to the console. [Try it](#)

- Accepts one parameter `s`: the message you would like to print

Example Usage:

```
# Example 1: Printing a string
print("Welcome to TIP101!") # Prints "Welcome to TIP101!" to the console

# Example 2: Printing an integer
print(100) # Prints 100 to the console

# Example 3: Printing a variable
s = "Welcome to CodePath!"
num = 7
print(s)    # Prints "Welcome to CodePath" to the console
print(num)  # Prints 7 to the console

# Example 4: Printing a List
lst = ["TIP101", "TIP102", "TIP103"]
print(lst)  # Prints ["TIP101", "TIP102", "TIP103"] to the console

# Example 5: Printing an expression
x = 5
y = 3
print(x + y) # Prints 8 to the console

# Example 6: Printing a function's return value

# Function my_func returns the value 6
```

```
def my_func():  
    return 6  
  
result = my_func()  
print(result) # Prints 6 to the console  
print(my_func(6)) # Prints 6 to the console (equivalent to above two lines)
```

Length

`len(s)` Returns the length of a list or string. [Try it](#)

- Accepts one parameter `s`: the list or string we would like to get the length of
- Returns the number of items in a list or the number of characters in a string

Example Usage:

```
# Example 1: Getting the Length of a List  
lst = ['a', 'b', 'c']  
lst_length = len(lst)  
print(lst_length) # Output: 3  
  
# Example 2: Getting the Length of string  
s = 'abcd'  
s_length = len(s)  
print(s_length) # Output: 4
```

Range

`range(start, stop, step)` Returns a sequence of numbers. [Try it](#)

- Accepts three parameters:
 - `start`: the first number in the sequence, this is an optional parameter. If we do not provide a `start`, the range will start from `0`.
 - `stop`: the last value in the sequence *exclusive*, meaning that the `stop` value is not actually included in the sequence. This is a *required* parameter

- `step`: how much to increment each number in the sequence. If we do not provide a `step`, each successive number in the sequence will increment by 1.

Example Usage:

```
# Example 1: Just the stop value
range(10) # Evaluates to the sequence: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

# Example 2: Start and stop value
range(1, 11) # Evaluates to the sequence: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

# Example 3: Start, stop, and step value
range(0, 30, 5) # Evaluates to the sequence: 0, 5, 10, 15, 20, 25
```

Conditionals

In Python, conditionals are defined using the `if`, `elif` and `else` keywords. The body of the conditional will execute if the expression placed after `if` or `elif` evaluates to `True`.

Example Usage:

```
# Example 1: Simple if statement
x = 3
if x > 1:
    print("This line will execute!")

if x > 5:
    print("This line will not execute!")

# Output: 'This line will execute!'

# Example 2: If-Elif statement

x = 10
y = 20

if x > y:
    print("x is greater than y")
elif y > x:
```

```
print("y is greater than x")
```

Output: 'y is greater than x'

Example 3: If-Elif-Else statement

```
x = 20
```

```
y = 20
```

```
if x > y:
```

```
    print("x is greater than y")
```

```
elif y > x:
```

```
    print("y is greater than x")
```

```
else:
```

```
    print("x and y are equal")
```

Output: 'x and y are equal'

For Loops

`for num in nums` Iterates through every number in nums. [Try it](#)

For loops in Python allow you to iterate over a sequence (such as a list, string, or range) and execute a block of code multiple times. This is particularly useful for accessing each element in a list or executing a function multiple times.

Example Usage:

Example 1: Iterating over a List

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit) # Outputs each fruit on a new line
```

Example 2: Using a for Loop with a range

```
for i in range(5):
```

```
    print(i) # Prints numbers 0 to 4
```

List Methods & Syntax

Append Method

`lst.append(item)` Adds an item to the end of the list. [Try it](#)

- Accepts one parameter `item`: the item you would like to add to the list

Example Usage:

```
# Example 1: Add an integer to the List
lst = [1, 2, 3, 4]
lst.append(5)
print(lst) # Prints [1, 2, 3, 4, 5]
```

Sort Method

`lst.sort()` Sorts the list in ascending order. [Try it](#)

- Does not have any required parameters
- Does not return any value

Example Usage:

```
# Example 1: List of integers
lst = [4, 2, 1, 3]
lst.sort()
print(lst) # Prints [1, 2, 3, 4]

# Example 2: List of strings
lst = ['b', 'a', 'd', 'c']
lst.sort()
print(lst) # Prints ['a', 'b', 'c', 'd']
```

List Indexing

`lst[index]` Accesses the element at position `index` in the list. [Try it](#)

- Uses **zero-based indexing**: the first element is at index `0`
- Accepts both positive and negative indices:
 - Positive indices count from the start (`0` is first)
 - Negative indices count from the end (`-1` is last)
- Accessing an index that does not exist will cause an **IndexError**

Example Usage:

```
# Example 1: Positive indexing
fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # Prints "apple"
print(fruits[2]) # Prints "cherry"

# Example 2: Negative indexing
print(fruits[-1]) # Prints "cherry"
print(fruits[-2]) # Prints "banana"

# Example 3: IndexError
# print(fruits[3]) # Causes IndexError: List index out of range
```

List Slicing

`lst[start:end]` Returns a new list containing elements from index `start` up to (but not including) `end`.

[Try it](#)

- `start` is optional — defaults to the beginning of the list
- `end` is optional — defaults to the end of the list
- Negative indices count from the end of the list
- The original list is **not** modified

Example Usage:

```
# Example 1: Start and end
nums = [0, 1, 2, 3, 4, 5]
```

```
print(nums[1:4]) # Prints [1, 2, 3]
```

Example 2: Start omitted

```
print(nums[:3]) # Prints [0, 1, 2]
```

Example 3: End omitted

```
print(nums[3:]) # Prints [3, 4, 5]
```

Example 4: Negative indices

```
print(nums[-3:]) # Prints [3, 4, 5]
```

Close Section

▼ Bonus Concepts Not In-Scope

The following syntax is nice to know and may improve your code readability or help you solve certain problems more easily. However, they are not *required* to solve any of the problems in this unit. These are **not in scope for the Unit 1 assessment**, and you do not need to memorize them! Click on the function to read more about how to use it.

- `lst.insert(x, item)` Inserts `item` into list at index `x`
- `lst.remove(item)` Removes `item` from list
- `lst.pop(x)` Removes element at index `x` from list
- `lst.copy()` Creates a copy of a list
- `abs(x)` Returns the absolute value of a number `x`.
- `enumerate()` Returns indices and values of list as pairs. Helpful for looping over indices and values of a list simultaneously!

Close Section

